Session 2.1: Web Data Extraction: Scraping Financial and Competitor Intelligence with Beautiful Soup

THE BUSINESS SITUATION

What Are Competitors Charging Today?

- Our pricing team watches 3 competitor websites, where prices sit on web pages, not in CSV files
- Copying hundreds of prices by hand is slow and error-prone
- The pricing manager asks: what does each competitor charge, and who discounts most?

Today, Python reads a web page and turns it into a table.

Today's flow: 60 min lecture, then 120 min hands-on lab

EXPECTED OUTCOMES

By the end of today, you will be able to:

1. ***"How does a web page reach my computer?"***
   Explain an HTTP GET request and read HTML
   HTTP: HyperText Transfer Protocol; HTML: HyperText Markup Language, the text with tags that describes a page
2. ***"What is BetaMart charging?"***
   Extract prices with Beautiful Soup and CSS selectors
   CSS (Cascading Style Sheets) selector: an address for tags, such as `.price`
3. ***"What if a product has no stock tag?"***
   Handle a missing tag and fix an `AttributeError`
   Missing tag: the search returns `None` (nothing found)

Session Map: Follow the Numbers

Today's question: *What does each competitor charge, and who discounts most?*

1 Why it matters
Lecture, part 1

1. Session Purpose
2. Learning Outcomes
3. Business Context: Why Scrape?
4. Data Basics: Web Request to HTML

→

2 Learn the tools
Lecture, part 2

5. Lab Practice Dataset
6. Keywords
7. Core Foundations
8. IDE Workflow
9. Code Examples + Quick Checks
10. Official References

→

3 Practice
Lab

11. Exercise 1: Guided - Competitor Price Benchmark
12. Exercise 2: Applied - Scrape Stock Levels Safely
13. Exercise 3: Advanced - Detect a Price Change

Why Scrape? Competitor Prices Live on Web Pages

By hand

- Open each competitor page
- Copy each price into Excel
- Do it all again tomorrow

3 sites × 100 products = 300 copy-pastes a day, with typos

VS

With a Python script

- Reads every product on the page in seconds
- Gives the same result every run
- Runs again every morning

Output: a clean table, ready for pandas

RULES OF THE ROAD: SCRAPE RESPONSIBLY

Public pages only, never behind a login

Check `robots.txt`, the site's list of pages scripts may read, and its terms of use

Pause between requests: `time.sleep(2)` waits 2 seconds

Send a User-Agent: a label that tells the site who is asking

From Web Request to HTML

Browser or Python script

1. HTTP GET request: "send me this page" →

← 2. Response: the page as HTML text

Web server `betamart.example`

WHAT THE SERVER SENDS: HTML TEXT

```
<div class="product" data-sku="SKU-801">
  <h3 class="name">Office Desk Chair</h3>
  <span class="price">$49.50</span>
  <span class="stock">120 units</span>
</div>
```

Python reads this text. `data-sku` holds the SKU (Stock Keeping Unit), the product code.

→

WHAT THE BROWSER DRAWS

Office Desk Chair

$49.50

120 units

People see this page

Three Files for Today

SAVED WEB PAGE 1

`session_02_01_catalog.html`

- Fictional BetaMart page, 4 product cards
- Tags: `div.product`, `.name`, `.price`, `.stock`
- SKU-807 has no stock tag (on purpose)
- Used in: Demos 1-2, Exercise 2

SAVED WEB PAGE 2

`session_02_01_gamma.html`

- Fictional GammaGlobal page, 4 products
- Different tags: `li.item`, `.title`, `.list-price`, `.sale-price`
- Products not on sale have no `.sale-price`
- Used in: Exercise 3

SCRAPED DATA (CSV)

`session_02_01_dataset.csv`

- 12 rows: 3 competitors × 4 categories
- `List_Price_USD`, `Promotion_Discount_Pct`, `Stock_Availability_Units`
- Used in: Demo 3, Exercises 1 and 3

All three files are in the `data/` folder. Every name and price is fictional teaching data.

Six Terms You Need Today

REQUEST

HTTP GET

What: a HyperText Transfer Protocol (HTTP) request that asks a server for a page

Why: every page your browser or script reads starts with one

PAGE TEXT

HTML Tag

What: an HTML (HyperText Markup Language) label around content, such as `<span class="price">`

Why: tags tell you where each price or name sits

STRUCTURE

DOM Tree

What: Document Object Model: tags nested inside tags, like folders

Why: one product card holds its name, price, and stock

TOOL

Beautiful Soup (`bs4`)

What: a Python library that reads HTML into a searchable tree

Why: find tags by name or class in one line

ADDRESS

CSS Selector

What: a Cascading Style Sheets (CSS) address for tags: `.price` = class "price"

Why: `.select()` / `.select_one()` use it to pick tags

RESPONSIBILITY

Ethical Scraping

What: follow `robots.txt`, pause between requests, stay on public pages

Why: protects the website and your company

How the Six Terms Fit Together

STEP 1: HTML TEXT FROM THE GET REQUEST

```
<div class="product">
  <h3 class="name">Office Desk Chair</h3>
  <span class="price">$49.50</span>
  <span class="stock">120 units</span>
</div>
... 3 more product cards
```

`BeautifulSoup()` →

STEP 2: DOM TREE

```
div.catalog
  div.product SKU-801
    h3.name
    span.price
    span.stock
  div.product × 3 more
```

STEP 3: CSS SELECTOR PICKS THE VALUES

`.select(".price")` →

price text
$49.50
$320.00
$429.00
$195.00

`float()` →

price_usd
49.5
320.0
429.0
195.0

Yellow in Steps 1-2: the tag the selector `.price` matches. Green: numbers pandas can calculate with

How Scraping Works

1. TEXT TO TABLE

From Tags to Rows and Columns

- A web page is text with tags, not a table
- Scraping picks out the values and lines them up as rows and columns

2. REPEATED CARDS

One Product Card = One Row

- Every product sits in the same kind of tag: `div.product`
- A `for` loop visits each card and builds one row

3. SELECT BY CLASS

Use Names, Not Positions

- "The 3rd tag" breaks if the site adds a tag before it
- `.price` follows the class name, so it is more likely to keep working
- Still recheck after a site redesign

4. MISSING TAGS

Nothing Found = `None`

- `.select_one()` returns `None` when a tag is missing
- `None.text` stops the script with an `AttributeError`
- Check first: `if tag is not None:`

Scraping, Seen Up Close

1-2. FOUR CARDS BECOME FOUR ROWS

`div.product SKU-801`
`div.product SKU-804`
`div.product SKU-807`
`div.product SKU-810`

→ `for`

sku	name	price_usd
SKU-801	Office Desk Chair	49.5
SKU-804	Wireless Router	320.0
SKU-807	Compact Refrigerator	429.0
SKU-810	Cordless Drill	195.0

3. POSITION BREAKS, CLASS HOLDS

Tag order	Before	After a SALE badge is added
1st	h3.name	h3.name
2nd	**span.price**	**span.badge "SALE"**
3rd	span.stock	**span.price**

"2nd tag" now returns "SALE"; `.price` still finds the price

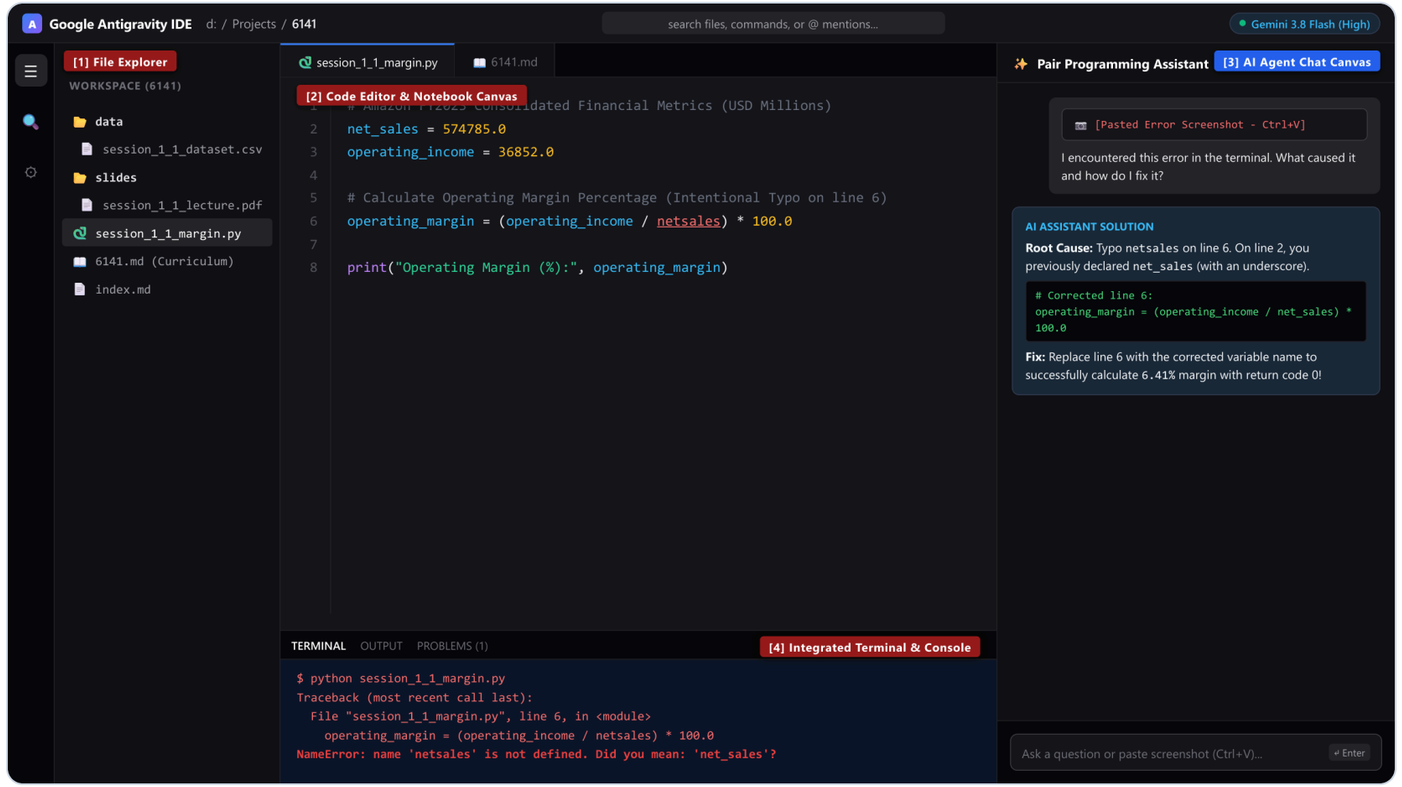
4. A MISSING TAG RETURNS `None`

sku	.select_one(".stock")
SKU-801	120 units
SKU-807	*None*

```
None.text  # AttributeError
if tag is not None:  # check first
```

The SKU-807 card on our page has no stock tag

Build, Test, and Diagnose in the IDE



Example: Terminal `[4]` shows a `NameError`; AI Chat `[3]` explains the fix

`[1]` File Explorer

Name it `YourName_S21_demo_1.py`
Example: `Reacher_S21_demo_1.py`
Check the dataset is in `data/`

`[2]` Code Editor

Write a few lines, run, repeat
Small steps keep errors small

`[3]` AI Agent Chat

Error? Capture the traceback (Python's error report) with `Win + Shift + S`
Paste with `Ctrl + V`, ask why

`[4]` Terminal

Run `python` + your script name
Return code `0` = no errors
First time only:
`pip install beautifulsoup4`

Read a Web Page with Beautiful Soup

YourName_S21_demo_1.py — Input → Process → Output

```
# 1. Input: read the saved web page (HTML text)
from bs4 import BeautifulSoup
html_text = open("data/session_02_01_catalog.html").read()

# 2. Process: build the tree, then find the first product name
soup = BeautifulSoup(html_text, "html.parser")
first_name = soup.find("h3").text

# 3. Output: show what we found
print("Page title:", soup.title.text)
print("First product:", first_name)
```

1. INPUT

`open().read()`

- Reads the saved page as one long text
- No `encoding=` needed: the file has only plain English characters
- On a live site, `requests.get(url).text` gives the same text (not run in class)

2. PROCESS

`BeautifulSoup()` and `.find()`

- `"html.parser"`: the built-in HTML reader
- `.find("h3")`: the first `h3` tag
- `.text`: the words inside it

3. OUTPUT

`soup.title.text`

The page title, then the first product

Find Every Match with `.find_all()`

NEW TOOL

`.find_all()`

What: returns **every** matching tag as a list; `.find()` returns only the first

Why: count how many products the page shows

Method	Result for "h3"
`.find()`	`1 tag: Office Desk Chair`
`.find_all()`	**`every h3 tag`**

YOUR TASK

Add One Line

In `YourName_S21_demo_1.py`, add one line to **3. Output**: print how many product names the page has.

Now

```
# (new line goes here)
```

Your change: fill in the yellow blank

```
print("Products on page:", len(soup._____("h3")))
```

Check: the Terminal shows `Products on page: 4`.

Turn Product Cards into Rows

YourName_S21_demo_2.py — Input → Process → Output

```
# 1. Input: read and parse the saved web page
from bs4 import BeautifulSoup
html_text = open("data/session_02_01_catalog.html").read()
soup = BeautifulSoup(html_text, "html.parser")

# 2. Process: one dictionary per product card
products = []
for card in soup.select(".product"):
    price_text = card.select_one(".price").text
    products.append({
        "sku": card["data-sku"],
        "name": card.select_one(".name").text.strip(),
        "price_usd": float(price_text.replace("$", "")),
    })

# 3. Output: one line per product
for item in products:
    print(item)
```

1. INPUT

Same read and parse

The page text becomes `soup`

2. PROCESS

Loop over the cards

- `.select()`: every card; `.select_one()`: one tag inside it
- `card["data-sku"]`: a value stored in the tag
- `float()`: `"$49.50"` to `49.5`, after removing `$`

3. OUTPUT

One dictionary per product

4 lines, one per card

Clean Text with `.get_text(strip=True)`

NEW TOOL

`.get_text(strip=True)`

What: returns the words inside a tag, with spaces and line breaks removed from both ends

Why: one call does the work of `.text.strip()`

Code	Result
`.text`	`"↵Office Desk Chair↵"`
`.get_text(strip=True)`	**`"Office Desk Chair"`**

↵ = a line break inside the tag (illustration)

YOUR TASK

Change One Line

In `YourName_S21_demo_2.py`, part **2. Process**: read the name with `.get_text()`.

Now

```
        "name": card.select_one(".name").text.strip(),
```

Your change: fill in the yellow blank

```
        "name": card.select_one(".name").__________(strip=True),
```

Check: the output does not change: the first line is still `{'sku': 'SKU-801', 'name': 'Office Desk Chair', 'price_usd': 49.5}`.

From Scraped Prices to a Comparison Table

YourName_S21_demo_3.py — Input → Process → Output

```
# 1. Input: prices already scraped from 3 competitor sites
import pandas as pd
df = pd.read_csv("data/session_02_01_dataset.csv")

# 2. Process: price after the advertised discount
df["Effective_Price_USD"] = (
    df["List_Price_USD"] * (1 - df["Promotion_Discount_Pct"] / 100)
).round(2)
electronics = df.loc[df["Category_Name"] == "Electronics"]

# 3. Output: compare competitors, then save the table
print(electronics[["Competitor_Name", "List_Price_USD",
                   "Effective_Price_USD"]])
electronics.to_csv("data/electronics_prices.csv", index=False)
```

1. INPUT

12 scraped prices

3 competitors, 4 categories, saved as CSV

2. PROCESS

Effective price

- List price minus the discount: `299.99 × (1 - 10 / 100) = 269.99`
- `.loc[]` keeps the Electronics rows

3. OUTPUT

`.to_csv()`

- Saves the table as a new CSV file
- `index=False`: leave out the row labels

Find the Lowest Price with `.min()`

NEW TOOL

`.min()`

What: returns the smallest value in a column; `.max()` returns the largest

Why: answers "what is the lowest competitor price?" in one step

Effective_Price_USD
269.99
281.6
263.5

Which one does `.min()` return?

YOUR TASK

Add Two Lines

In `YourName_S21_demo_3.py`, add two lines to **3. Output**: print the lowest Electronics price after discount.

Now

```
# (new line goes here)
```

Your change: fill in the yellow blank

```
lowest = electronics["Effective_Price_USD"].____()
print("Lowest price:", lowest)
```

Check: the Terminal shows `Lowest price: 263.5` (GammaGlobal).

Where to Look Things Up

BEAUTIFUL SOUP 4

beautiful-soup-4.readthedocs.io

Every method used today: `.find()`, `.find_all()`, `.select()`, `.get_text()`

REQUESTS

requests.readthedocs.io

How `requests.get(url)` fetches a live page, and how to send a User-Agent

W3C HTML STANDARD

w3.org/TR/html52

The World Wide Web Consortium's official rules for HTML tags

Exercise 1 and Exercise 2

EXERCISE 1: GUIDED PRACTICE

Competitor Price Benchmark

- Create `YourName_S21_benchmark.py`
- Load `data/session_02_01_dataset.csv` and add `Effective_Price_USD`
- Group by `["Category_Name", "Competitor_Name"]`: average discount and effective price
- Find the deepest discount and the lowest price in Electronics and Hardware

EXERCISE 2: APPLIED PRACTICE

Scrape Stock Levels Safely

- In `YourName_S21_practice.py`, extend Demo 2 to read `.stock`
- Run it: SKU-807 has no stock tag, so read the `AttributeError` and ask AI Chat `[3]`
- Fix it with `if stock_tag is not None:`, else `"not shown"`
- Save the table with `.to_csv()` to `data/competitor_price_audit.csv`

Success check: Ex. 1: in Hardware, GammaGlobal has the deepest discount (15%) but AlphaRetail has the lowest price (`157.5`). Ex. 2: 4 rows, SKU-807 shows `not shown`.

Detect a Price Change on a New Site

WHAT TO BUILD: `YourName_S21_challenge.py`

1. Scrape `session_02_01_gamma.html`: `sku`, `name`, `List_Price_Web`, `Current_Price_Web` (sale price if the tag exists, else list price)
2. `Discount_Pct_Web` = (list - current) / list × 100, 1 decimal
3. Merge with the GammaGlobal rows of the CSV
4. `Discount_Changed` = web discount `!=` `Promotion_Discount_Pct`
5. Print, then save `data/gamma_price_check.csv`
6. Print `Products with a changed discount: N`

NEW TOOLS

- Key columns with different names: `left_on` for the first table, `right_on` for the second

```
pd.merge(
    web_df, gamma_df,
    left_on="sku", right_on="Product_SKU",
    how="left",
)
```

- `!=`: "not equal", one `True` / `False` per row

HINT

- `sale_tag = card.select_one(".sale-price")`, then `if sale_tag is not None:`

Self-check: only `SKU-811` (Electric Oven) changed: `15.0`% on the web vs `10.0`% in last week's CSV. Full steps: section 13 of the course page.